

AI 开发环境配置讲义

从零搭建：LaTeX 排版 + AI 编程 + MCP 工具链

四川农业大学经济学院数字经济系
肖诗顺 教授 | 供金融专业学生使用

2026 年 3 月

摘要

本讲义面向四川农业大学经济学院金融专业的学生，系统介绍如何从零搭建一套完整的 AI 辅助开发环境。内容涵盖：(1) Windows 系统下 MiKTeX 及 macOS 系统下 MacTeX 的 XeLaTeX 中文论文排版环境；(2) Node.js 与 Python 基础运行时；(3) Git 版本控制与 CNB 国产云开发平台；(4) AI IDE (Qoder) 与 AI CLI 工具 (QoderCLI、Claude Code、OpenAI Codex) 的安装与配置；(5) CC Switch 多账号/多服务商管理工具；(6) MCP (Model Context Protocol) 服务器的配置与使用；(7) OpenCLI / OpenClaw 等前沿 AI 工具链的实践。每一步均配有命令行示例和验证方法，力求“复制-粘贴即可复现”，同时提供 Windows 和 macOS 双平台指引。

目录

1 环境概览与前提条件	4
1.1 目标环境一览	4
1.2 硬件与系统要求	4
2 第一部分：LaTeX 排版环境	5
2.1 安装 MiKTeX (Windows)	5
2.2 安装后自动获得的工具	6
2.3 验证安装	6
2.4 测试中文编译	6
2.5 安装 MacTeX (macOS)	7
2.6 系统中文字体清单	9
3 第二部分：基础运行时环境	10
3.1 安装 Node.js	10
3.2 安装 Python	10

3.3	安装 Git	11
3.4	CNB 云开发平台——国产代码托管与协作	11
3.4.1	注册与配置	12
3.4.2	安装 CNB CLI 工具	12
3.4.3	将本地项目推送到 CNB	12
3.4.4	CNB 与 GitHub 的关系	13
3.4.5	CNB 专属 CLI 命令	13
3.5	安装常用 Python 包	14
4	第三部分：AI IDE——Qoder	15
4.1	什么是 Qoder?	15
4.2	安装 Qoder IDE	15
4.3	Qoder 核心功能	15
4.4	配置 MCP 服务器	15
5	第四部分：AI CLI 工具	16
5.1	QoderCLI——终端版 Qoder	16
5.2	Claude Code——Anthropic 终端 AI 编程助手	17
5.3	OpenAI Codex CLI	18
5.4	三大 AI CLI 对比	18
5.5	CC Switch——AI CLI 多账号管理神器	18
5.5.1	CC Switch 核心功能	19
5.5.2	安装 CC Switch	19
5.5.3	使用 CC Switch 管理 Claude Code	20
5.5.4	CC Switch 与 ccs witch.sh 账号切换器	20
6	第五部分：OpenCLI——把网站变成命令行	21
6.1	什么是 OpenCLI?	21
6.2	安装与配置	21
6.3	常用命令示例	22
7	第六部分：OpenClaw——AI Skill 管理	22
7.1	什么是 OpenClaw?	22
7.2	安装与使用	23
7.3	推荐安装的 Skill	23
8	第七部分：MCP 服务器配置详解	23
8.1	什么是 MCP?	23
8.2	MCP 启动方式	24

8.3	推荐配置的 MCP 服务器	24
8.4	完整 mcp.json 配置示例	24
9	第八部分：MCP / Skill / CLI 三种范式	25
10	第九部分：OpenMAIC——AI 多智能体互动课堂	26
10.1	项目简介	26
10.2	核心功能	26
10.3	方式一：在线使用（托管模式）	27
10.4	方式二：本地部署	27
10.4.1	前提条件	27
10.4.2	验证服务	29
10.5	方式三：Docker 部署	29
10.6	方式四：Vercel 云端部署	29
10.7	支持的 LLM 服务商	30
10.8	OpenClaw 集成	30
10.9	项目目录结构	31
11	第十部分：DeepTutor——AI 个性化学习助手	31
11.1	项目简介	31
11.2	核心功能	32
11.3	DeepTutor 与 OpenMAIC 的对比	32
11.4	方式一：Docker 部署（推荐）	32
11.5	方式二：手动安装（用于开发）	34
11.6	支持的 LLM 服务商	35
11.7	在研究中使用 DeepTutor	35
11.8	项目目录结构	35
12	第十一部分：完整安装检查清单	36
13	第十二部分：常见问题与故障排除	37
14	附录：推荐学习资源	39

1 环境概览与前提条件

1.1 目标环境一览

完成本讲义所有步骤后，你将拥有以下完整工具链：

类别	工具	Windows 版本	macOS 版本
操作系统	Windows / macOS	11 25H2	15 Sequoia+
LaTeX 排版	MiKTeX / MacTeX	25.12	2026
运行时	Node.js / Python	v24.x / 3.12.x	v24.x / 3.12.x
AI IDE	Qoder	最新版	最新版
AI CLI	QoderCLI / Claude Code / Codex	0.1.35 / 2.x / 0.x	同左
AI Skill	OpenClaw	2026.x	2026.x
Web CLI	OpenCLI	1.x	1.x

1.2 硬件与系统要求

Windows 系统

项目	要求
操作系统	Windows 11 家庭中文版（25H2 或更高版本）
语言版本	必须为中文版（自带完整中文字体，这是 XeLaTeX 编译中文的前提）
磁盘空间	至少预留 10 GB（MiKTeX + Node.js + Python + AI 工具）
内存	建议 8 GB 以上（AI 工具 + 浏览器同时运行）
网络	需要稳定的互联网连接（安装宏包、AI API 调用）

重要提示

如果使用英文版 Windows，需额外安装中文字体（宋体 SimSun、黑体 SimHei、楷体 KaiTi、仿宋 FangSong），否则 xelatex 编译中文会报错。

macOS 系统

项目	要求
操作系统	macOS 15 Sequoia 或更高版本 (Intel / Apple Silicon 均可)
中文支持	macOS 自带华文宋体、华文黑体、华文楷体等, XeLaTeX 可直接调用
磁盘空间	至少预留 12 GB (MacTeX 完整版约 7 GB + Node.js + Python + AI 工具)
内存	建议 8 GB 以上 (Apple Silicon 机型通常 16 GB 起步)
网络	需要稳定的互联网连接 (安装宏包、AI API 调用)

macOS 用户提示

macOS 无需额外安装中文字体, 系统自带的华文字体系列 (STSong、STHeiti、STKaiti、STFangsong) 即可满足 XeLaTeX 中文编译需求。建议安装 Homebrew 包管理器以简化后续软件安装。

2 第一部分: LaTeX 排版环境

2.1 安装 MiKTeX (Windows)

MiKTeX 是 Windows 上最推荐的 LaTeX 发行版, 支持按需自动安装宏包。

步骤 1: 下载与安装 MiKTeX

1. 访问 <https://miktex.org/download>
2. 下载 Basic MiKTeX Installer (约 230 MB)
3. 运行安装程序, 选择 "Install for current user" (用户级安装, 无需管理员权限)
4. 安装路径默认为: %LOCALAPPDATA%\Programs\MiKTeX\
5. 安装完成

步骤 2: 配置自动安装宏包

1. 打开 MiKTeX Console (开始菜单搜索即可)
2. 进入 Settings → 将 "Install missing packages" 设为 Always
3. 进入 Updates → 点击 "Check for updates" 更新一次

2.2 安装后自动获得的工具

工具	版本	用途
xelatex	4.16	中文论文必备（支持 Unicode+ 系统字体）
pdflatex	4.23	英文论文编译（最常用）
lualatex	—	现代引擎，部分模板需要
bibtex	—	传统参考文献处理
biber	2.21	现代参考文献处理（配合 biblatex）
latexmk	—	自动化多轮编译
texworks	—	MiKTeX 自带编辑器（备用）

2.3 验证安装

打开终端（Windows 使用 PowerShell，macOS 使用 Terminal），逐条运行以下命令，确认版本号正常输出：

```
xelatex --version
pdflatex --version
biber --version
```

2.4 测试中文编译

新建文件 `test.tex`，写入以下内容：

```
\documentclass{ctexart}
\begin{document}
你好，LaTeX！这是一个中文编译测试。
如果你能看到本页中文正常显示，说明环境配置成功。
\end{document}
```

在终端运行：

```
xelatex test.tex
```

如果生成 `test.pdf` 且中文显示正常，LaTeX 环境配置完成！

首次编译注意

首次运行 xelatex 时，MiKTeX 会自动下载缺失的宏包（如 ctex 系列），弹出安装提示时点击“Install”即可。首次编译耗时较长（1-2 分钟），后续编译约 5-10 秒。

2.5 安装 MacTeX (macOS)

MacTeX 是 macOS 上推荐的 TeX Live 完整发行版，包含所有常用宏包，无需按需下载。

方式一：通过 Homebrew 安装（推荐）

```
# 安装 Homebrew (如尚未安装)
/bin/bash -c "$(curl -fsSL
    https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# 安装 MacTeX (完整版, 约 7 GB)
brew install --cask mactex
```

下载速度提示

Homebrew 默认源下载 MacTeX 可能较慢（约 7 GB）。如果速度过慢，建议使用方式二的清华镜像手动安装。

方式二：通过清华 CTAN 镜像手动安装（速度更快）

```
# 下载 MacTeX 安装包 (约 7 GB, 清华镜像国内速度快)
curl -L -o /tmp/mactex.pkg \

    "https://mirrors.tuna.tsinghua.edu.cn/CTAN/systems/mac/mactex/mactex-20260324.pkg"
\
--progress-bar

# 安装 (需要管理员密码)
sudo installer -pkg /tmp/mactex.pkg -target /

# 安装完成后删除安装包释放空间
rm /tmp/mactex.pkg
```

关于 sudo

sudo 是“Super User DO”的缩写，表示以管理员权限执行命令。执行后系统会要求输入你的 Mac 登录密码（输入时屏幕不会显示字符，这是正常的），输入完成后按回车即可。

方式三：官网下载.pkg 安装

1. 访问 <https://www.tug.org/mactex/>
2. 下载 MacTeX.pkg（约 7 GB）
3. 双击.pkg 文件，按安装向导完成安装

配置 PATH 环境变量

MacTeX 安装后，可执行文件位于 `/usr/local/texlive/2026/bin/universal-darwin`。通常安装程序会自动配置 PATH，如果终端找不到 `xelatex` 命令，需手动添加：

```
# 将以下内容添加到 ~/.zshrc (macOS 默认 shell 为 zsh)
echo 'export PATH="/usr/local/texlive/2026/bin/universal-darwin:$PATH"' >>
~/.zshrc

# 使配置生效
source ~/.zshrc
```

验证 MacTeX 安装

```
xelatex --version    # 应输出 XeTeX 3.141592653-2.6-0.999998 (TeX Live 2026)
pdflatex --version
biber --version
```

测试中文编译 (macOS)

新建文件 `test.tex`，写入以下内容（与 Windows 相同）：

```
\documentclass{ctexart}
\begin{document}
你好，LaTeX! 这是一个中文编译测试。
如果你能看到本页中文正常显示，说明环境配置成功。
\end{document}
```

在终端 (Terminal) 运行：

```
xelatex test.tex
```

如果生成 `test.pdf` 且中文显示正常，macOS LaTeX 环境配置完成！

MacTeX vs MiKTeX

MacTeX 是 TeX Live 的 macOS 封装版，安装包含**全部**宏包（约 7 GB），无需像 MiKTeX 那样按需下载。优点是安装后即可直接使用所有宏包，缺点是初始安装体积较大。

2.6 系统中文字体清单

Windows 中文字体

以下字体为 Windows 中文版自带，xelatex 可直接调用：

字体	LaTeX 调用名	用途
宋体	SimSun	正文
黑体	SimHei	标题
楷体	KaiTi	引用、批注
仿宋	FangSong	公文、声明
微软雅黑	Microsoft YaHei	现代风格

macOS 中文字体

macOS 系统自带以下中文字体，XeLaTeX 可直接调用：

字体	LaTeX 调用名	用途
华文宋体	STSong	正文
华文黑体	STHeiti	标题
华文楷体	STKaiti	引用、批注
华文仿宋	STFangsong	公文、声明
苹方	PingFang SC	现代风格（macOS 专属）

ctex 宏包自动适配

使用 `ctexart` 文档类时，ctex 宏包会**自动检测**操作系统并选择对应的中文字体，无需手动指定。Windows 下自动使用 SimSun/SimHei，macOS 下自动使用 STSong/STHeiti。

3 第二部分：基础运行时环境

3.1 安装 Node.js

Node.js 是所有 AI CLI 工具（QoderCLI、Claude Code、OpenCLI 等）的运行基础。

Windows 安装步骤

1. 访问 <https://nodejs.org/>，下载 LTS 版本（推荐 v22.x 或 v24.x）
2. 运行安装程序，全部勾选默认选项
3. 安装完成后验证：

```
node --version    # 应输出 v22.x.x 或 v24.x.x
npm --version     # 应输出 10.x.x 或更高
```

macOS 安装步骤

```
# 方式一：通过 Homebrew（推荐）
brew install node

# 方式二：通过官网下载 .pkg 安装包
# 访问 https://nodejs.org/ 下载 macOS 版 LTS 安装包

# 验证
node --version    # 应输出 v22.x.x 或 v24.x.x
npm --version     # 应输出 10.x.x 或更高
```

注意

不要选“Current”版本（不稳定），始终选 LTS（长期支持版）。

3.2 安装 Python

Python 是数据分析、机器学习和部分 MCP 服务器的基础。

Windows 安装步骤

1. 访问 <https://www.python.org/downloads/>，下载 Python 3.12.x
2. 安装时务必勾选“Add Python to PATH”
3. 验证：

```
python --version    # 应输出 Python 3.12.x  
pip --version       # 应输出 pip 24.x
```

macOS 安装步骤

```
# 方式一：通过 Homebrew（推荐）  
brew install python  
  
# 方式二：通过官网下载 .pkg 安装包  
# 访问 https://www.python.org/downloads/ 下载 macOS 版  
  
# 验证（macOS 上可能需要使用 python3）  
python3 --version    # 应输出 Python 3.12.x  
pip3 --version       # 应输出 pip 24.x
```

macOS Python 提示

macOS 自带的 Python 版本较旧，建议通过 Homebrew 安装最新版本。安装后使用 python3 和 pip3 命令（或创建别名 alias python=python3）。

3.3 安装 Git

Git 是版本控制和克隆 GitHub 项目的必备工具。

```
# Windows：下载地址 https://git-scm.com/download/win  
# macOS：通过 Homebrew 安装  
brew install git  
  
# 或者首次在终端运行 git 命令时，macOS 会自动提示安装 Xcode Command Line  
#   Tools（含 Git）  
  
# 验证（Windows / macOS 通用）  
git --version    # 应输出 git version 2.x.x
```

3.4 CNB 云开发平台——国产代码托管与协作

CNB（CodeNotebook，<https://cnb.cool>）是国内领先的云原生代码托管与开发平台，提供 Git 代码托管、CI/CD 流水线、AI 辅助编程等功能。相比 GitHub，CNB 具有国内访问速度快、中文界面友好、集成 AI Skill 等优势，推荐金融专业同学使用。

3.4.1 注册与配置

1. 访问 <https://cnb.cool>，使用微信扫码注册/登录
2. 登录后进入个人设置，创建**访问令牌**（Access Token）：
 - 进入「设置」→「访问令牌」→「创建新令牌」
 - 勾选 `repo-code:rw` 权限
 - 记录生成的 Token（仅显示一次）

3.4.2 安装 CNB CLI 工具

```
# 安装 CNB CLI (全局)
npm install @cnbcool/cnb-cli -g

# 安装 Skills 工具 (AI 增强功能)
npm install skills -g

# 验证安装
cnb --version
skills list

# 添加 CNB Skill (增强 IDE 中的 AI 功能)
npx skills add https://cnb.cool/cnb/skills/cnb-skill.git --agent codebuddy
-y --copy
```

3.4.3 将本地项目推送到 CNB

```
# 1. 进入项目目录
cd your-project-folder

# 2. 初始化 Git 仓库 (如未初始化)
git init

# 3. 创建 .gitignore (排除不需要的文件)
echo "*.lock" > .gitignore
echo ".DS_Store" >> .gitignore

# 4. 添加并提交
git add .
```

```
git commit -m "Initial commit"

# 5. 配置 CNB 远程仓库（替换 your-token 为你的令牌）
git remote add origin
    https://cnb:your-token@cnb.cool/your-org/your-repo.git

# 6. 推送代码
git push -u origin main
```

3.4.4 CNB 与 GitHub 的关系

维度	GitHub	CNB
服务器位置	海外	国内
访问速度	可能较慢/不稳定	快速稳定
Git 命令	标准 Git	完全相同
仓库地址格式	github.com/user/repo	cnb.cool/org/repo
认证方式	SSH Key / Token	Token（微信登录获取）
AI 功能	Copilot（付费）	内置 AI Skill（免费）
CI/CD	GitHub Actions	CNB Pipeline

重要提示

CNB 的 Git 核心命令（clone、add、commit、push、pull、branch 等）与 GitHub 完全一致。如果你已经会用 Git，切换到 CNB 几乎零学习成本——只是仓库地址和认证方式不同。

3.4.5 CNB 专属 CLI 命令

除了标准 Git 命令外，CNB 还提供平台管理命令：

```
# 登录 CNB
cnb login

# 查看组织列表
cnb organizations list-top-groups

# 查看仓库列表
cnb repositories list-repos
```

```
# Issue 管理
cnb issues list-issues

# PR 管理
cnb pulls list-pulls

# AI 功能——自动总结 PR
cnb ai summarize-pr
```

3.5 安装常用 Python 包

以下是金融/经济学研究常用的核心包，在终端中运行（Windows 使用 PowerShell, macOS 使用 Terminal）：

```
# 基础科学计算（macOS 用户将 pip 替换为 pip3）
pip install numpy pandas scipy scikit-learn statsmodels

# 可视化
pip install matplotlib seaborn plotly

# 金融数据源
pip install yfinance tushare openbb

# AI / 大模型
pip install openai transformers torch

# 文档处理
pip install python-docx python-pptx openpyxl pdfplumber

# Jupyter 环境
pip install jupyter jupyterlab ipython

# MCP 相关
pip install mcp fastmcp
```

4 第三部分：AI IDE——Qoder

4.1 什么是 Qoder?

Qoder 是一款基于 VS Code 架构的 AI 智能体编程平台，内置多模型支持（GPT-4、Claude、DeepSeek 等），提供代码补全、Agent 对话、MCP 工具集成等功能。

4.2 安装 Qoder IDE

安装步骤

1. 访问 <https://qoder.com/download>
2. 下载对应系统版本：
 - Windows: Qoder-Setup.exe (约 150 MB)
 - macOS: Qoder.dmg (约 200 MB)
3. Windows 双击运行安装向导；macOS 拖拽 Qoder 到 Applications 文件夹
4. 首次启动后，点击右上角用户图标 → 登录/注册 Qoder 账号

4.3 Qoder 核心功能

功能	说明
智能补全	上下文感知的代码自动补全
Agent 对话	右侧面板与 AI 对话，可读写文件、运行命令
MCP 集成	内置 MCP 工具管理器，可连接数据库、API、Office 等
Skill 系统	加载 SKILL.md 文件赋予 AI 领域知识
多模型切换	支持 GPT-4o、Claude Sonnet、DeepSeek 等模型

4.4 配置 MCP 服务器

Qoder 的 MCP 配置文件位于：

```
# Windows
%APPDATA%\Qoder\SharedClientCache\extension\local\mcp.json

# macOS
```

```
~/Library/Application
  Support/Qoder/SharedClientCache/extension/local/mcp.json
```

在 Qoder 中配置 MCP 的步骤：

1. 打开 Qoder → 点击左下角齿轮图标 → 搜索“MCP”
2. 或直接编辑上述 mcp.json 文件
3. 每个 MCP Server 的配置格式如下：

```
{
  "mcpServers": {
    "fetch": {
      "command": "uvx",
      "args": ["mcp-server-fetch"]
    },
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp@latest"]
    }
  }
}
```

5 第四部分：AI CLI 工具

5.1 QoderCLI——终端版 Qoder

QoderCLI 是 Qoder IDE 的命令行版本，可在终端中直接与 AI 交互。

安装步骤（Windows / macOS 通用）

1. 确保 Node.js 已安装（见第二部分）
2. 在终端中运行（Windows 使用 PowerShell，macOS 使用 Terminal）：

```
npm install -g @qoder-ai/qodercli
```

验证安装：

```
qoder --version # 应输出版版本号，如 0.1.35
```

使用方法

```
# 进入项目目录后启动
cd your-project-folder
qoder

# QoderCLI 会自动读取项目结构，进入交互式对话
# 你可以直接用自然语言描述需求，例如：
# > 帮我写一个读取 CSV 并绘制折线图的 Python 脚本
```

5.2 Claude Code——Anthropic 终端 AI 编程助手

Claude Code 是 Anthropic 公司推出的命令行 AI 编程工具。

安装步骤

1. 安装：

```
npm install -g @anthropic-ai/claude-code
```

2. 验证：

```
claude --version # 应输出版本号，如 2.1.50
```

3. 首次启动需要配置 API Key：

```
claude
# 按提示登录 Anthropic 账号或输入 API Key
# 支持 OAuth 登录（推荐）或直接输入 ANTHROPIC_API_KEY
```

Claude Code 系统要求

- Windows 10 1809+ / Windows Server 2019+ / macOS 12 Monterey+
- Node.js 18+
- 4 GB+ RAM
- 需要互联网连接

5.3 OpenAI Codex CLI

OpenAI 推出的终端 AI 编程助手。

```
# 安装
npm install -g @openai/codex

# 验证
codex --version    # 应输出版本号

# 首次使用需配置 OpenAI API Key
# Windows PowerShell:
$env:OPENAI_API_KEY = "sk-your-api-key-here"
# macOS / Linux Terminal:
export OPENAI_API_KEY="sk-your-api-key-here"
```

5.4 三大 AI CLI 对比

工具	厂商	安装命令	核心模型
QoderCLI	Qoder AI	npm i -g @qoder-ai/qodercli	多模型切换
Claude Code	Anthropic	npm i -g @anthropic-ai/claude-code	Claude Sonnet
Codex CLI	OpenAI	npm i -g @openai/codex	GPT-4o

5.5 CC Switch——AI CLI 多账号管理神器

当你同时使用 Claude Code、Codex、Gemini CLI 等多个 AI 工具时，频繁切换 API 服务商和账号非常繁琐。CC Switch 是一款跨平台桌面应用，提供可视化界面统一管理所有 AI CLI 工具的配置。

5.5.1 CC Switch 核心功能

功能	说明
多工具管理	统一管理 Claude Code、Codex、Gemini CLI、Open-Claw 等 7 款工具
一键切换	50+ 预置服务商，点击即可切换 API 提供方，无需手动编辑配置文件
系统托盘快切	系统托盘右键直接切换服务商，无需打开完整界面
统一 MCP 管理	一个面板管理所有工具的 MCP 服务器配置，支持双向同步
Skill 管理	一键从 GitHub 安装 Skill，跨工具同步
用量追踪	实时追踪 API 花费、请求次数和 Token 用量
云端同步	通过 Dropbox/OneDrive/iCloud/WebDAV 跨设备同步配置

5.5.2 安装 CC Switch

macOS 安装（推荐 Homebrew）

```
# 通过 Homebrew 安装
brew install --cask cc-switch

# 更新
brew upgrade --cask cc-switch
```

Windows 安装

1. 访问 <https://github.com/farion1231/cc-switch/releases>
2. 下载最新版 CC-Switch-v*.msi 安装包
3. 双击运行安装向导

其他方式

- macOS 也可从 Releases 页面下载 .dmg 文件手动安装
- 官方网站: <https://ccswitch.io>

5.5.3 使用 CC Switch 管理 Claude Code

步骤 1：首次启动

安装后启动 CC Switch，首次运行会自动检测已安装的 AI CLI 工具并导入其现有配置。

步骤 2：添加服务商

1. 点击”Add Provider”（添加服务商）
2. 从 50+ 预置模板中选择，或自定义配置
3. 填入 API Key 和 Base URL
4. 保存即可

步骤 3：切换服务商

```
# 方式一：主界面点击 "Enable" 启用指定服务商
# 方式二：系统托盘右键 → 直接选择服务商名称（即时生效）
# 方式三：切换后重启终端即可使用新配置
# 注意：Claude Code 支持热切换，无需重启
```

步骤 4：回到官方登录

如需切换回 Anthropic 官方账号登录：

1. 在 CC Switch 中添加一个”Official Login”预置
2. 切换到该配置
3. 在终端运行 `claude` 后按提示完成 OAuth 登录

5.5.4 CC Switch 与 ccswitch.sh 账号切换器

除了 CC Switch 桌面应用外，还有一个轻量级命令行工具 `ccswitch.sh`，专门用于在多个 Claude Code 账号之间快速切换（适合有多个 Anthropic 订阅的场景）：

```
# 安装（下载脚本）
curl -O
    https://raw.githubusercontent.com/ming86/cc-account-switcher/main/ccswitch.sh
chmod +x ccswitch.sh

# 添加当前登录的账号
./ccswitch.sh --add-account
```

```
# 列出所有已保存账号
./ccswitch.sh --list

# 切换到下一个账号
./ccswitch.sh --switch

# 切换到指定账号
./ccswitch.sh --switch-to 2
./ccswitch.sh --switch-to user2@example.com

# 切换后需重启 Claude Code 生效
```

推荐方案

- **CC Switch 桌面应用**（推荐）：适合需要管理多个 AI 工具、多个服务商的场景，提供可视化界面
- **ccswitch.sh 脚本**：适合仅需在多个 Claude Code 官方账号间切换的轻量需求

两者可以共存使用，互不冲突。

6 第五部分：OpenCLI——把网站变成命令行

6.1 什么是 OpenCLI?

OpenCLI 可以将任意网站转换为命令行工具，复用 Chrome 浏览器的登录态，让 AI 直接操作网页。

6.2 安装与配置

安装步骤

1. 全局安装：

```
npm install -g @jackwener/opencli
```

2. 验证安装：

```
opencli --version # 应输出 1.1.1 或更高
```

3. 安装 Chrome 浏览器扩展 (Browser Bridge):

- 前往 <https://github.com/jackwener/opencli/releases> 下载 opencli-extension.zip
- 在 Chrome 中访问 chrome://extensions/
- 开启右上角**开发者模式**
- 点击”加载已解压的扩展程序” → 选择解压后的文件夹

4. 运行 setup 验证连接:

```
opencli setup
# 应输出:
# Daemon is running on port 19825
# Chrome extension connected
# Setup complete!
```

6.3 常用命令示例

```
# Google 搜索
opencli google search "农村金融" --lang zh --limit 10

# 探测网站结构
opencli explore https://www.example.com

# 级联策略访问
opencli cascade https://some-website.com/api/data

# 查看健康状态
opencli doctor
```

7 第六部分：OpenClaw——AI Skill 管理

7.1 什么是 OpenClaw?

OpenClaw 是一个 AI Skill 包管理工具，类似 npm 之于 Node.js，它管理 SKILL.md 文件——教 AI 如何完成特定领域任务的知识文档。

7.2 安装与使用

安装步骤

```
# 全局安装
npm install -g openclaw

# 验证
openclaw --version    # 应输出 2026.x.x

# 安装 Skill（以学术论文分析 Skill 为例）
openclaw install academic-paper-analysis
openclaw install scientific-writing
openclaw install arxiv
```

7.3 推荐安装的 Skill

Skill 名称	类别	用途
arxiv	文献检索	搜索、下载 arXiv 论文
research-lit	文献检索	文献综述与分析
paper-write	论文写作	按章节起草 LaTeX 论文
paper-compile	论文编译	编译 LaTeX 为 PDF
scientific-writing	学术写作	IMRAD 格式学术论文写作
paper-slides	演示	生成 Beamer 演讲幻灯片
grant-proposal	基金申请	撰写基金申请书
scikit-learn	机器学习	ML 算法参考与代码生成
networkx	网络分析	图论与网络分析

8 第七部分：MCP 服务器配置详解

8.1 什么是 MCP?

MCP (Model Context Protocol) 是 Anthropic 于 2024 年底发布的 AI 工具连接协议，让 AI 助手能够访问外部数据源和工具（数据库、API、Office 文档、浏览器等）。

方式	命令前缀	适用场景
npx	Node.js 包	Chrome DevTools, Playwright, Excel 等
uvx	Python 包 (via uv)	Fetch, PPT, Word, CNKI 等
本地 exe	直接运行	WindowsMCP.Net 桌面自动化

8.2 MCP 启动方式

前提条件

使用 uvx 需先安装 uv (Python 包管理器):

```
# Windows
pip install uv

# macOS
pip3 install uv
# 或通过 Homebrew
brew install uv
```

8.3 推荐配置的 MCP 服务器

#	服务名	安装/启动命令	用途
1	fetch	uvx mcp-server-fetch	HTTP 网页抓取
2	playwright	npx @playwright/mcp@latest	浏览器自动化
3	excel	npx @negokaz/excel-mcp-server	Excel 读写
4	ppt	uvx ppt_mcp_server	PPT 编辑
5	word	uvx word_mcp_server	Word 文档编辑
6	cnki	uvx cnki-mcp	中国知网检索
7	chrome-devtools	npx chrome-devtools-mcp@latest	Chrome 调试
8	amap-maps	npx @amap/amap-maps-mcp-server	高德地图

8.4 完整 mcp.json 配置示例

以下为推荐的 Qoder MCP 配置文件:

```
{
  "mcpServers": {
```

```
"fetch": {
  "command": "uvx",
  "args": ["mcp-server-fetch"]
},
"playwright": {
  "command": "npx",
  "args": ["@playwright/mcp@latest"]
},
"excel": {
  "command": "npx",
  "args": ["@negokaz/excel-mcp-server"]
},
"ppt": {
  "command": "uvx",
  "args": ["ppt_mcp_server"]
},
"word": {
  "command": "uvx",
  "args": ["word_mcp_server"]
},
"cnki": {
  "command": "uvx",
  "args": ["cnki-mcp"]
}
}
```

9 第八部分：MCP / Skill / CLI 三种范式

理解这三种 AI 接口范式的区别，是高效使用 AI 工具的关键。

维度	MCP	Skill	CLI
本质	连接协议	领域知识文件	Shell 命令
类比	USB-C 接口	操作手册	万能遥控器
形态	JSON-RPC Server	Markdown 文件	Shell 命令
开发成本	高（需写 Server）	低（写 Markdown）	零
适合场景	企业多租户 SaaS	知识沉淀与复用	本地自动化

黄金组合

CLI（执行层）+ Skill（知识层）+ MCP（连接层）= 完整的 AI 工具链。
日常个人使用以 CLI + Skill 为主，MCP 在需要跨系统数据连接时介入。

10 第九部分：OpenMAIC——AI 多智能体互动课堂

10.1 项目简介

OpenMAIC（Open Multi-Agent Interactive Classroom）是清华大学 THU-MAIC 团队开发的开源 AI 互动课堂平台。它能将任何主题或文档转化为丰富的互动学习体验——由 AI 教师和 AI 同学进行语音讲解、白板绘图，并与你展开实时讨论。

项目	详情
GitHub 地址	https://github.com/THU-MAIC/OpenMAIC
在线体验	https://open.maic.chat/
开源协议	AGPL-3.0
技术栈	Next.js16、React19、TypeScript5、LangGraph1.1、Tailwind CSS 4
论文	JCST'26: From MOOC to MAIC (DOI: 10.1007/s11390-025-6000-0)

10.2 核心功能

功能	说明
幻灯片讲解	AI 教师配合聚光灯、激光笔动画进行语音讲解
测验问答	单选/多选/简答题，AI 实时判分与反馈
交互式模拟	基于 HTML 的物理模拟器、流程图等可视化实验
项目制学习	选择角色，与 AI 智能体协作完成结构化项目 (PBL)
白板	AI 实时绘制图表、推导公式、流程图讲解
圆桌讨论	多个不同人设的 AI 围绕话题展开辩论
语音交互	语音合成 (TTS) 讲解 + 麦克风语音识别输入
导出功能	下载可编辑的.pptx 幻灯片或交互式.html 网页
网络搜索	智能体在课堂中实时搜索网络获取最新信息
国际化	界面支持中文和英文

10.3 方式一：在线使用（托管模式）

最快体验 OpenMAIC 的方式，无需任何本地部署。

在线使用步骤

1. 打开浏览器访问 <https://open.maic.chat/>
2. 在首页输入想学的主题，或上传 PDF 文档
3. 点击生成——AI 将在几分钟内创建完整课堂
4. 两阶段生成流程如下：
 - 第一阶段——大纲生成：AI 分析输入，生成结构化课堂大纲
 - 第二阶段——场景生成：每个大纲条目转化为幻灯片、测验、模拟实验或 PBL 活动
5. 生成完成后进入课堂，开始互动学习

金融研究场景示例

- ”用 20 分钟教我资本资产定价模型（CAPM）”
- ”分析农村金融与农业现代化的关系”
- ”逐步讲解面板数据回归方法”
- 上传研究论文 PDF 后说”帮我解读这篇论文并教给我”

10.4 方式二：本地部署

完全掌控数据、支持离线运行、自由配置模型。

10.4.1 前提条件

- Node.js ≥ 20 （第二部分已安装）
- pnpm ≥ 10 （Node.js 包管理器）
- Git（第二部分已安装）

步骤 1：安装 pnpm

```
# 安装 pnpm (如尚未安装)
npm install -g pnpm

# 验证
pnpm --version    # 应输出 10.x.x 或更高
```

步骤 2: 克隆并安装依赖

```
# 克隆仓库
git clone https://github.com/THU-MAIC/OpenMAIC.git
cd OpenMAIC

# 安装所有依赖 (约需 2-5 分钟)
pnpm install
```

步骤 3: 配置环境变量

```
# 复制示例配置文件
cp .env.example .env.local
```

用任意文本编辑器打开 `.env.local`，至少填写一个 LLM 服务商 API Key：

```
# 方案 A: OpenAI
OPENAI_API_KEY=sk-...

# 方案 B: Anthropic
ANTHROPIC_API_KEY=sk-ant-...

# 方案 C: Google Gemini (推荐, 质量与速度最佳平衡)
GOOGLE_API_KEY=...

# 方案 D: DeepSeek (高性价比国产模型)
DEEPSEEK_API_KEY=sk-...

# 方案 E: 国内服务商

GLM_API_KEY=...                # 智谱 GLM
GLM_BASE_URL=https://open.bigmodel.cn/api/paas/v4
MINIMAX_API_KEY=sk-cp-...      # MiniMax
MINIMAX_BASE_URL=https://api.minimax.chat/v1
```

推荐模型

Gemini 3 Flash 效果与速度的最佳平衡。追求最高质量可选 Gemini 3.1 Pro（速度较慢）。

设置默认模型：

```
DEFAULT_MODEL=google:gemini-3-flash-preview
```

步骤 4：启动开发服务器

以开发模式启动（支持热更新）

```
pnpm dev
```

在浏览器中打开 <http://localhost:3000> 即可开始使用！

步骤 5（可选）：生产环境构建

构建优化的生产版本

```
pnpm build
```

启动生产服务器

```
pnpm start
```

10.4.2 验证服务

健康检查（应返回 JSON，包含 status "ok"）

```
curl http://localhost:3000/api/health
```

10.5 方式三：Docker 部署

确保已安装 Docker，然后执行：

```
cd OpenMAIC
```

```
cp .env.example .env.local
```

编辑 .env.local 填入 API Key，然后：

```
docker compose up --build
```

服务将在 <http://localhost:3000> 可用。数据持久化在 openmaic-data 卷中。

10.6 方式四：Vercel 云端部署

零基础设施的云端部署方案：

1. 在 GitHub 上 Fork <https://github.com/THU-MAIC/OpenMAIC>
2. 导入到 <https://vercel.com/new>
3. 配置环境变量（至少一个 LLM API Key）
4. 一键部署——Vercel 自动处理构建和托管

10.7 支持的 LLM 服务商

服务商	环境变量	示例模型
OpenAI	OPENAI_API_KEY	GPT-4o, GPT-4o-mini
Anthropic	ANTHROPIC_API_KEY	Claude 3.5 Sonnet/Haiku
Google	GOOGLE_API_KEY	Gemini 3 Flash, Gemini 3.1 Pro
DeepSeek	DEEPSEEK_API_KEY	DeepSeek-V3, DeepSeek-R1
智谱 GLM	GLM_API_KEY	GLM-5, GLM-4.7
MiniMax	MINIMAX_API_KEY	MiniMax-M2.5
通义千问	QWEN_API_KEY	Qwen 系列
Grok (xAI)	GROK_API_KEY	Grok 系列

10.8 OpenClaw 集成

OpenMAIC 可通过 OpenClaw 在聊天应用（飞书、Slack、Telegram 等）中直接控制：

```
# 安装 OpenMAIC Skill
clawhub install openmaic

# 或手动复制 Skill
mkdir -p ~/.openclaw/skills
cp -R /path/to/OpenMAIC/skills/openmaic ~/.openclaw/skills/openmaic
```

两种使用模式：

- 托管模式——在 <https://open.maic.chat/> 获取访问码，无需本地部署
- 本地部署模式——Skill 引导你逐步完成 clone、配置和启动

10.9 项目目录结构

```
OpenMAIC/
  app/                                # Next.js App Router (18+ 个 API 端点)
    api/generate/                     # 场景生成流水线
    api/chat/                         # 多智能体讨论 (SSE 流式传输)
    api/pbl/                          # 项目制学习端点
    classroom/[id]/                  # 课堂回放页面
  lib/                                # 核心业务逻辑
    generation/                      # 两阶段课堂生成流水线
    orchestration/                   # LangGraph 多智能体编排
    playback/                        # 回放状态机
    action/                          # 28+ 种动作执行引擎
  components/                        # React UI 组件
  skills/openmaic/                   # OpenClaw Skill (SKILL.md + 参考文档)
  .env.local                         # 你的 API 密钥 (从 .env.example 创建)
```

11 第十部分：DeepTutor——AI 个性化学习助手

11.1 项目简介

DeepTutor 是香港大学数据科学实验室 (HKUDS) 开发的开源 AI 个性化学习助手。它将文档知识问答、交互式可视化、习题生成和深度研究功能整合为一体——由多智能体协作与 RAG（检索增强生成）技术驱动。

项目	详情
GitHub 地址	https://github.com/HKUDS/DeepTutor
官方网站	https://hkuds.github.io/DeepTutor/
Star 数	10,800+（截至 2026 年初）
开源协议	AGPL-3.0
技术栈	Python3.10+ (FastAPI 后端)、Next.js16、React19、Tailwind CSS
后端架构	FastAPI + LlamaIndex + RAGAnything + 知识图谱

11.2 核心功能

功能	说明
文档知识问答	上传教材、论文、手册；多智能体协同解题，提供精确引用
交互式学习可视化	将复杂概念转化为可视化图解、分步解析和交互演示
习题生成器	针对你的知识水平生成定制化练习题；模仿真实考试风格
深度研究	结合 RAG、网络搜索和学术论文数据库进行系统性主题探索
创意生成	自动化头脑风暴与交互式协作写作，支持播客生成
知识库	从 PDF/TXT/MD 文件构建和管理个人知识库
个人笔记本	基于会话的上下文记忆，支持持续学习
代码执行	在沙箱环境中运行 Python 代码完成计算任务
语音朗读	通过 OpenAI 或 Azure 语音引擎进行文本转语音输出
多语言支持	完整的中英文界面支持

11.3 DeepTutor 与 OpenMAIC 的对比

维度	OpenMAIC	DeepTutor
定位	互动课堂（幻灯片、测验、PBL、白板）	个性化学习助手（知识问答、研究、习题）
输入	主题文本或 PDF → 完整互动课堂	上传文档 → 知识库
智能体	AI 教师 + AI 同学（多角色）	解题器、研究者、习题生成器、创意生成器（多智能体）
后端架构	Next.js + LangGraph（全 JS）	Python FastAPI + Next.js 前端
最佳场景	生成和展示互动课程	深度学习、研究探索和个性化练习

11.4 方式一：Docker 部署（推荐）

最简单的运行方式——无需安装 Python 或 Node.js。

步骤 1：克隆并配置

```
# 克隆仓库
git clone https://github.com/HKUDS/DeepTutor.git
cd DeepTutor

# 创建环境配置
cp .env.example .env
```

用文本编辑器打开 `.env`，填写必需的变量：

```
# LLM 配置（必填）
LLM_BINDING=openai
LLM_MODEL=gpt-4o
LLM_API_KEY=sk-你的API密钥
LLM_HOST=https://api.openai.com/v1

# Embedding 配置（必填）
EMBEDDING_BINDING=openai
EMBEDDING_MODEL=text-embedding-3-small
EMBEDDING_API_KEY=sk-你的API密钥
EMBEDDING_HOST=https://api.openai.com/v1
EMBEDDING_DIMENSION=3072

# 国内方案：使用 DeepSeek（性价比高）
# LLM_BINDING=deepseek
# LLM_MODEL=deepseek-chat
# LLM_API_KEY=sk-你的DeepSeek密钥
# LLM_HOST=https://api.deepseek.com/v1
```

步骤 2：使用 Docker 启动

```
# 方式 A：从源码构建
docker compose up

# 方式 B：使用预构建镜像（更快）
docker run -d --name deeptutor `
  -p 8001:8001 -p 3782:3782 `
  --env-file .env `
  -v ${PWD}/data:/app/data `
  -v ${PWD}/config:/app/config:ro `
```

```
ghcr.io/hkuds/deeptutor:latest
```

在浏览器中访问 <http://localhost:3782>。API 文档位于 <http://localhost:8001/docs>。

常用 Docker 命令

```
docker compose up -d      # 后台启动
docker compose down       # 停止所有服务
docker compose logs -f    # 查看实时日志
docker compose up --build # 代码修改后重新构建
```

11.5 方式二：手动安装（用于开发）

步骤 1：搭建 Python 环境

```
# 使用 conda（推荐）
conda create -n deeptutor python=3.10
conda activate deeptutor

# 或使用 venv
python -m venv venv
# Windows 激活：
venv\Scripts\activate
# macOS / Linux 激活：
source venv/bin/activate
```

步骤 2：安装依赖

```
# 一键安装（推荐）
python scripts/install_all.py

# 或手动安装
pip install -r requirements.txt
npm install --prefix web
```

步骤 3：启动

```
# 同时启动前后端
python scripts/start_web.py

# 或分别启动：
```

```
# 后端: python src/api/run_server.py
# 前端: cd web && npm run dev -- -p 3782
```

在浏览器中访问 <http://localhost:3782> 即可使用。

11.6 支持的 LLM 服务商

服务商	LLM_BINDING 值	示例模型
OpenAI	openai	GPT-4o, GPT-4o-mini
Azure OpenAI	azure_openai	GPT-4o (Azure 托管)
Anthropic	anthropic	Claude 3.5 Sonnet
DeepSeek	deepseek	DeepSeek-V3, DeepSeek-Chat
Groq	groq	LLaMA-3, Mixtral
Together AI	together	多种开源模型
Ollama (本地)	ollama	LLaMA-3, Phi-3, Qwen
LM Studio	lm_studio	任意 GGUF 模型

11.7 在研究中使用 DeepTutor

DeepTutor 研究 workflow

1. **构建知识库**: 将所有课程教材、参考论文、课堂笔记以 PDF/TXT/MD 格式上传
2. **智能问答**: 使用问答模块提问特定概念——DeepTutor 会从你的文档中提供精确引用的答案
3. **生成习题**: 使用习题生成器创建与即将到来的考试风格匹配的练习题
4. **深度研究**: 跨文档和网络资源探索主题; 生成文献综述摘要
5. **创意生成**: 使用创意生成模块与 AI 协作者一起头脑风暴研究假设

11.8 项目目录结构

```
DeepTutor/
  src/                # Python 后端源代码
    agents/          # 多智能体模块 (解题器、研究者等)
    api/              # FastAPI REST 端点
    knowledge/        # RAG 流水线与知识库管理
```

```
services/      # LLM、Embedding、TTS 服务集成
tools/         # 网络搜索、代码执行、PDF 解析
web/           # Next.js 16 前端
app/           # 页面与路由
components/    # React UI 组件
config/        # YAML 配置文件 (agents.yaml、main.yaml)
data/          # 知识库、用户数据、日志
scripts/       # 安装与启动脚本
.env           # 你的 API 密钥 (从 .env.example 创建)
```

12 第十一部分：完整安装检查清单

安装完成后，在终端中逐条运行以下命令进行验证（Windows 使用 PowerShell，macOS 使用 Terminal）：

```
# 1. LaTeX 环境
xelatex --version
biber --version

# 2. 运行时
node --version
npm --version
python --version
pip --version
git --version

# 3. AI CLI 工具
qoder --version
claude --version
codex --version
opencli --version
openclaw --version
cnb --version

# 4. OpenCLI 健康检查
opencli doctor

# 5. OpenMAIC 本地部署检查
cd /path/to/OpenMAIC
```

```
pnpm dev
# 访问 http://localhost:3000 确认页面正常加载

# 6. DeepTutor 本地部署检查
cd /path/to/DeepTutor
python scripts/start_web.py
# 访问 http://localhost:3782 确认页面正常加载

# 7. Python 核心包检查
python -c "import numpy; print(numpy.__version__)"
python -c "import pandas; print(pandas.__version__)"
python -c "import openai; print(openai.__version__)"
```

验证标准

以上所有命令均应正常输出版本号，无报错。如果某项缺失，回到对应章节重新安装。

13 第十二部分：常见问题与故障排除

问题	解决方法
xelatex 编译中文报错	确认使用的是 Windows 中文版系统；检查是否安装了 SimSun、SimHei 等字体
MiKTeX 首次编译卡住	MiKTeX 在按需下载宏包，等待下载完成即可；确保网络畅通
fancyhdr 警告	在导言区添加 <code>\setlength{\headheight}{14pt}</code>
npm install -g 失败	检查 Node.js 是否已安装；尝试以管理员权限运行 PowerShell
OpenCLI 扩展未连接	运行 <code>opencli setup</code> ；确保 Chrome 已启动且扩展已加载
pip install 速度慢	使用国内镜像： <code>pip install -i https://pypi.tuna.tsinghua.edu.cn/simple xxx</code>

问题	解决方法
Python 不在 PATH 中	重新安装 Python 并勾选"Add Python to PATH"; 或手动添加环境变量
Claude Code API Key	运行 <code>claude</code> 后按提示登录或设置 <code>ANTHROPIC_API_KEY</code> 环境变量
pnpm install 失败	确保 <code>Node.js</code> ≥ 20 且 <code>pnpm</code> ≥ 10 ; 先运行 <code>npm install -g pnpm</code> ; 删除 <code>node_modules</code> 后重试
OpenMAIC 页面空白	检查 <code>.env.local</code> 中是否填写了至少一个 LLM API Key; 查看终端错误信息
OpenMAIC 生成课堂失败	确认 API Key 有效; 检查服务商余额和速率限制; 尝试更换 <code>DEFAULT_MODEL</code>
DeepTutor docker compose up 失败	确保 Docker 已安装并运行; 检查 <code>.env</code> 中的 <code>LLM_API_KEY</code> 和 <code>EMBEDDING_API_KEY</code> 是否有效; 尝试 <code>docker compose build --no-cache</code>
DeepTutor 知识库上传失败	检查终端日志; 确保 <code>EMBEDDING_MODEL</code> 与 <code>EMBEDDING_DIMENSION</code> 匹配 (如 <code>text-embedding-3-small</code> 对应 3072)
DeepTutor 3782 端口无法访问	确认后端 (8001) 和前端 (3782) 端口未被占用; 查看 <code>docker compose logs -f</code> 获取错误信息
macOS 终端找不到 <code>xelatex</code>	MacTeX 安装后需配置 <code>PAT</code> 在 <code>~/.zshrc</code> 中添加 <code>export PATH="/usr/local/texlive/2026/bin/universal-darwin:\$PATH"</code> , 然后运行 <code>source ~/.zshrc</code>
macOS 下载 MacTeX 太慢	使用清华 CTAN 镜像下载: <code>curl -L -o /tmp/mactex.pkg "https://mirrors.tuna.tsinghua.edu.cn/CTAN/systems/mac/mactex"</code>
macOS 安装 <code>.pkg</code> 需要密码	<code>sudo installer</code> 需要管理员权限, 输入 Mac 登录密码即可 (输入时屏幕不显示字符是正常的)
macOS <code>npm install -g</code> 权限不足	运行 <code>sudo npm install -g xxx</code> , 或使用 <code>nvm</code> 管理 <code>Node.js</code> 避免权限问题
macOS <code>python</code> 命令找不到	Homebrew 安装的 Python 使用 <code>python3</code> 和 <code>pip3</code> 命令; 可在 <code>~/.zshrc</code> 中添加 <code>alias python=python3</code>

14 附录：推荐学习资源

- CNB 云开发平台: <https://cnb.cool>
- CNB CLI 文档: <https://docs.cnb.cool/>
- OpenMAIC GitHub: <https://github.com/THU-MAIC/OpenMAIC>
- OpenMAIC 在线体验: <https://open.maic.chat/>
- OpenMAIC 论文 (JCST'26) : <https://jcst.ict.ac.cn/en/article/doi/10.1007/s11390-025-6000-0>
- DeepTutor GitHub: <https://github.com/HKUDS/DeepTutor>
- DeepTutor 官方网站: <https://hkuds.github.io/DeepTutor/>
- DeepTutor 预构建 Docker 镜像: <https://github.com/HKUDS/DeepTutor/pkgs/container/deeptutor>
- MiKTeX 官方文档: <https://miktex.org/howto>
- MacTeX 官方网站: <https://www.tug.org/mactex/>
- 清华 CTAN 镜像 (MacTeX 下载) : <https://mirrors.tuna.tsinghua.edu.cn/CTAN/systems/mac/mactex/>
- LaTeX 入门教程: <https://tug.ctan.org/info/install-latex-guide-zh-cn/>
- Homebrew 官方网站: <https://brew.sh/>
- Qoder 快速开始: <https://docs.qoder.com/zh/quick-start>
- QoderCLI 文档: <https://docs.qoder.com/zh/cli/quick-start>
- Claude Code 设置: <https://code.claude.com/docs/zh-CN/setup>
- OpenCLI GitHub: <https://github.com/jackwener/opencli>
- OpenClaw 教程: <https://github.com/xianyu110/awesome-openclaw-tutorial>
- MCP 协议文档: <https://modelcontextprotocol.io/>

本讲义由四川农业大学经济学院数字经济系肖诗顺教授编写，供金融专业学生使用。

版本 2.0 | 2026 年 3 月